

Retention Priority Ordering Fix — Diagnosis & Implementation

Date: 2026-08-11 **Signal under investigation:** 7de8d9c7 (`entry_not_opened`)
User: Leonardo 82756f8c **Broker:** fcabb782 (disconnected during the whole incident window)

The problem

We need the important logs to stay in the database so we can debug trades. We don't care about the spam logs. The database was doing the opposite: keeping the spam and deleting the important logs.

This is what actually happened on Aug 10. Leonardo sent a trade signal (7de8d9c7). It failed with `entry_not_opened`. When we tried to debug *why* it failed, there was nothing in the logs table — no `pipeline_summary`, no `handle_start/handle_end`, no trace at all. It looked like the pipeline silently vanished. It didn't. The pipeline ran fine and wrote its logs — a bug in the cleanup job destroyed them within minutes.

Fix 1 — keep the right logs (the database change)

Background: The worker runs a cleanup job every 10 minutes. For each user, it keeps the 500 most recent log rows and deletes the rest. This stops the logs table from growing forever.

Some log rows are important — like `pipeline_summary` and `handle_start/handle_end`. These tell us what the pipeline did with a signal. Other rows are just noise — like the `auto_be` failure rows that got written every half second.

The cleanup job was *supposed* to give the important rows priority so they'd never be deleted. But the code had a one-word bug.

The sort instruction said: *rank priority rows last, rank noise rows first*. That means when a user's rows exceeded 500, the important rows — the ones ranked last — got deleted **first**. The noise survived.

The fix changes one word: `ASC` → `DESC`. Now it means: *rank priority rows first, rank noise rows last*. Result: important rows always survive; noise rows get deleted first.

This matches what actually happened with Leonardo: he had 724 spam rows (from the disconnected broker). The important rows — like `pipeline_summary` for signal 7de8d9c7 — were deleted within 10 minutes. Now they won't be.

Fix 2 — stop writing so much spam (the worker change)

Background: The auto-BE monitor is a background job. It watches open trades that have “auto break-even” turned on. When the trade hits a profit level, the job moves the stop-loss to break-even. That’s a real feature that protects profits.

But during the incident, Leonardo’s broker was disconnected. Every tick, the job tried to move the stop-loss and failed. Every failure wrote a log row. The job ticks about every half second, so that’s roughly 2 failed log rows per second — the 724 rows.

The fix adds a cool-down: for each trade, the job only writes a failure log row once every 5 minutes. The job still *tries* to move the stop-loss on every tick — nothing about the actual trading changes. We only stopped writing the same failure message over and over.

In short

- **Fix 1** stops the cleanup job from deleting our important logs.
- **Fix 2** stops the worker from writing thousands of spam rows in the first place.

Both fix the same mess from two directions: less spam going in, and important rows staying in.

Status

- **Fix 1 — migration:** written and **applied to staging** (HTTP 201, verified DESC live). Migration file: `supabase/migrations/20260811100000_fix_retention_priority_ordering`. Prod **not touched** (pending go-ahead).
- **Fix 2 — worker throttle:** code edited, typechecks clean. File: `worker/src/autoManagementMonitor.ts` (`FAILURE_LOG_THROTTLE_MS`, default 5 min, env `AUTO_BE_FAILURE_LOG_THROTTLE_MS`).

Note on migration bookkeeping

The `schema_migrations.statements` column is a Postgres `text[]`, not JSON. The initial registration attempt used the wrong literal format and was skipped as non-fatal — the fix itself is unaffected. Registration needs a proper array literal.